HALALAN REVAMP

System Requirements

Ayson, Aaron Eldrich
Elizarde, Samantha Coleen
Manzano, John Anthony

Client: University Computer Center

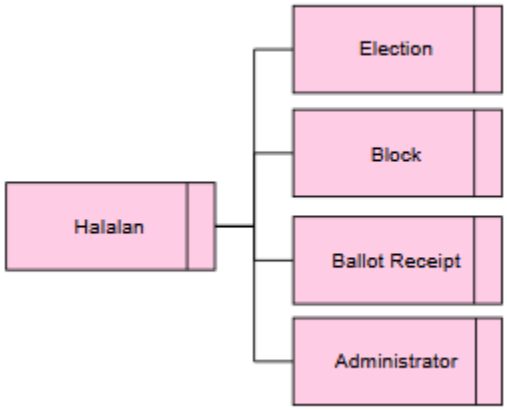
https://cs191-ame.github.io/blog/

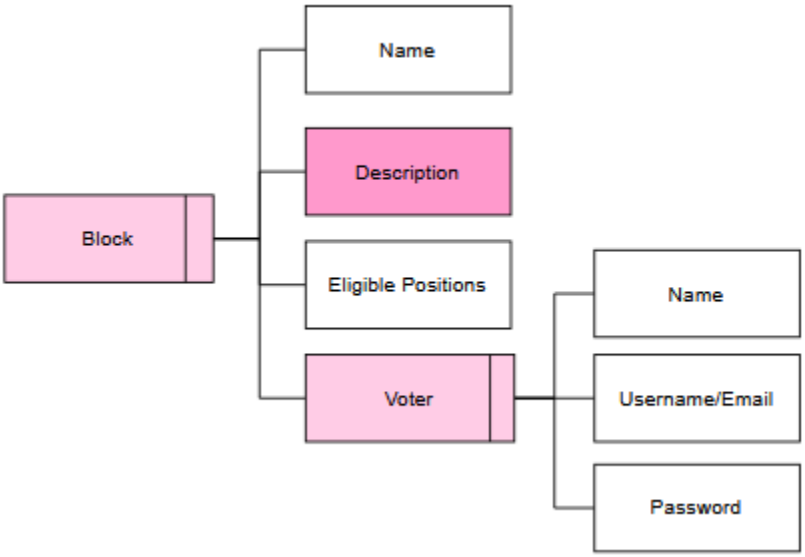
CS 191 WFR1
Prof. Ligaya Leah Figueroa

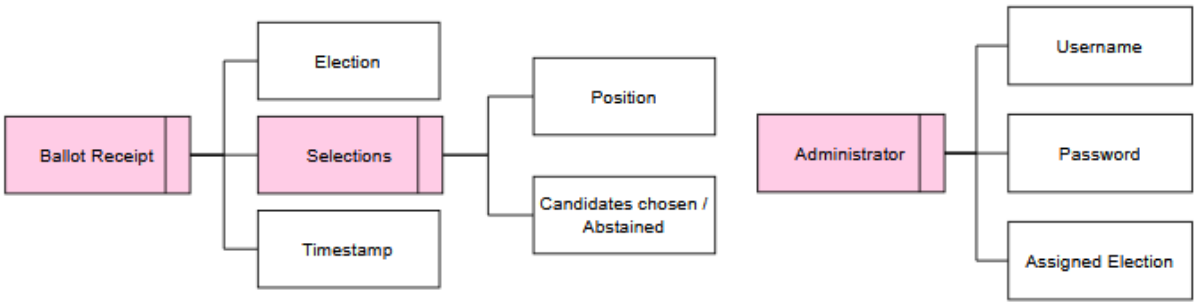
1. Web App Requirements Model

1.1 Content Model

The data tree below shows the content model for the Halalan WebApp, with the main content objects Election, Block, Ballot Receipt, and Administrator. The notation follows that of Pressman's Figure 7.10 (Chapter 7, Section 7.5.4, "Content Model for WebApps"). Data items are drawn as unshaded rectangles, content objects as shaded rectangles, and composite items as lighter shaded rectangles marked by a line near the right edge. The first figure shows the top level, and the figures that follow expand each branch.






Ballot Receipt
Election
Selections
Timestamp
Position
Candidates chosen / Abstained
Administrator
Username
Password
Assigned Election


Election
Election ID
Name
Results visibility
Status
Parent Election
Position
Party
Results
Statistics
Candidate
Title
Description
Maximum votes
Ordinality
Abstain allowed
Name
Description
Photo
Party affiliation
Alias
Votes per candidate
Abstentions
Name
Logo
Description
Alias
Voters who voted
Registered voters
Voting duration

1.2 Interaction Model

Actors and Goals

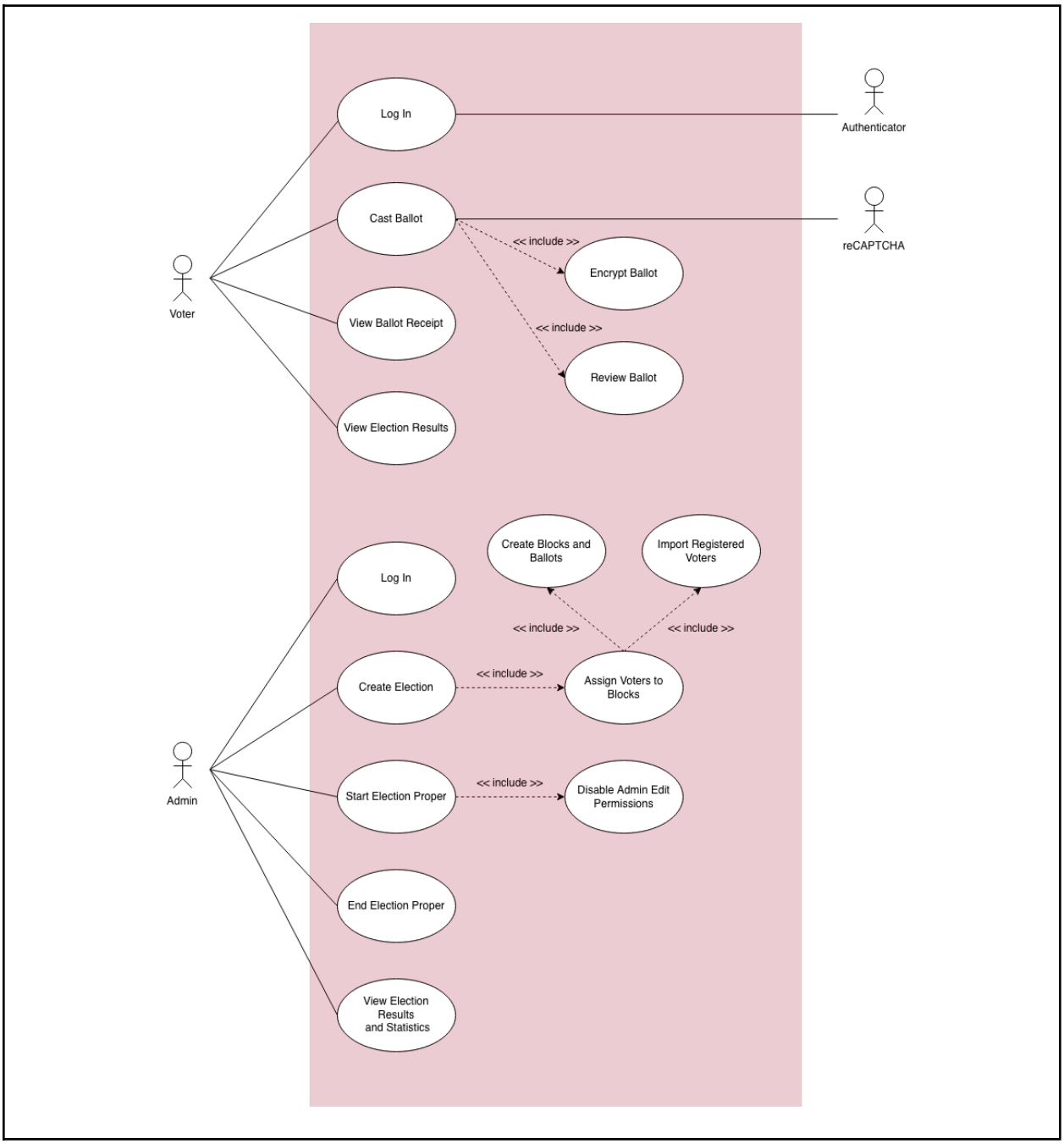
For this election system, 3 main actors have been identified: the administrators (admins), the voters, and external authenticators. The administrators and the voters are primary actors, while the external authenticators are secondary actors.

The admins are individuals in charge of managing the elections. The main admins are personnel from the University of the Philippines Computer Center. They will be responsible for the creation of the elections, for compiling and uploading into the system all relevant, and overseeing the election proper. For the university-wide elections, admin status may be extended to select individuals from each college or department so that they may manage the start and duration of their respective elections. They are considered primary actors as they will be initiating the various events that occur in the system.

The voters consist of members of the university that are eligible and registered to vote in the elections. During the election proper, they are to first, login to the system using their university-issued email address (UP email) and have their identity verified using 2-factor authentication. Then, they may fill out their ballots by selecting candidates. Once they are ready to submit and their vote has been validated by the system, they will be presented with a summary of their vote alongside a prompt to finalize their ballot. Upon submission, they will need to successfully answer a reCAPTCHA for the system to fully process their cast ballot. Voters are a primary actor as well since their actions drive the interactions with the system.

Authenticators such as Google Authenticator and Google reCAPTCHA on the other hand are secondary or participating actors in the system as their services or actions are only triggered as a response to the actions of a primary actor, particularly the voters.

Use Cases



VOTER.

Log In. A voter can log into the system using their UP email. After inputting their credentials, the system will send a request to the authenticator (Google Authenticator) for 2-factor

authentication. Once the voter successfully accomplishes this, they will be logged into the system.

Cast Ballot. During the election proper, a voter can fill out their ballot and cast it. Some subroutines of this case include a display of a summary of their ballot to allow them to review it prior to casting, and the encryption of the ballot, handled by the backend (CodeIgniter). Finally, the submission of the ballot will also send a request for a reCAPTCHA. Only once the voter passes the reCAPTCHA will their vote be stored in the database.

View Ballot Receipt. Following their voting, a voter may log into the site in order to view their ballot receipt containing a summary of their vote.

View Election Results. A voter may track the results of the election in real time by viewing the election results on the site.

ADMIN.

Log In. An admin account may log into the system in order to edit or manage elections.

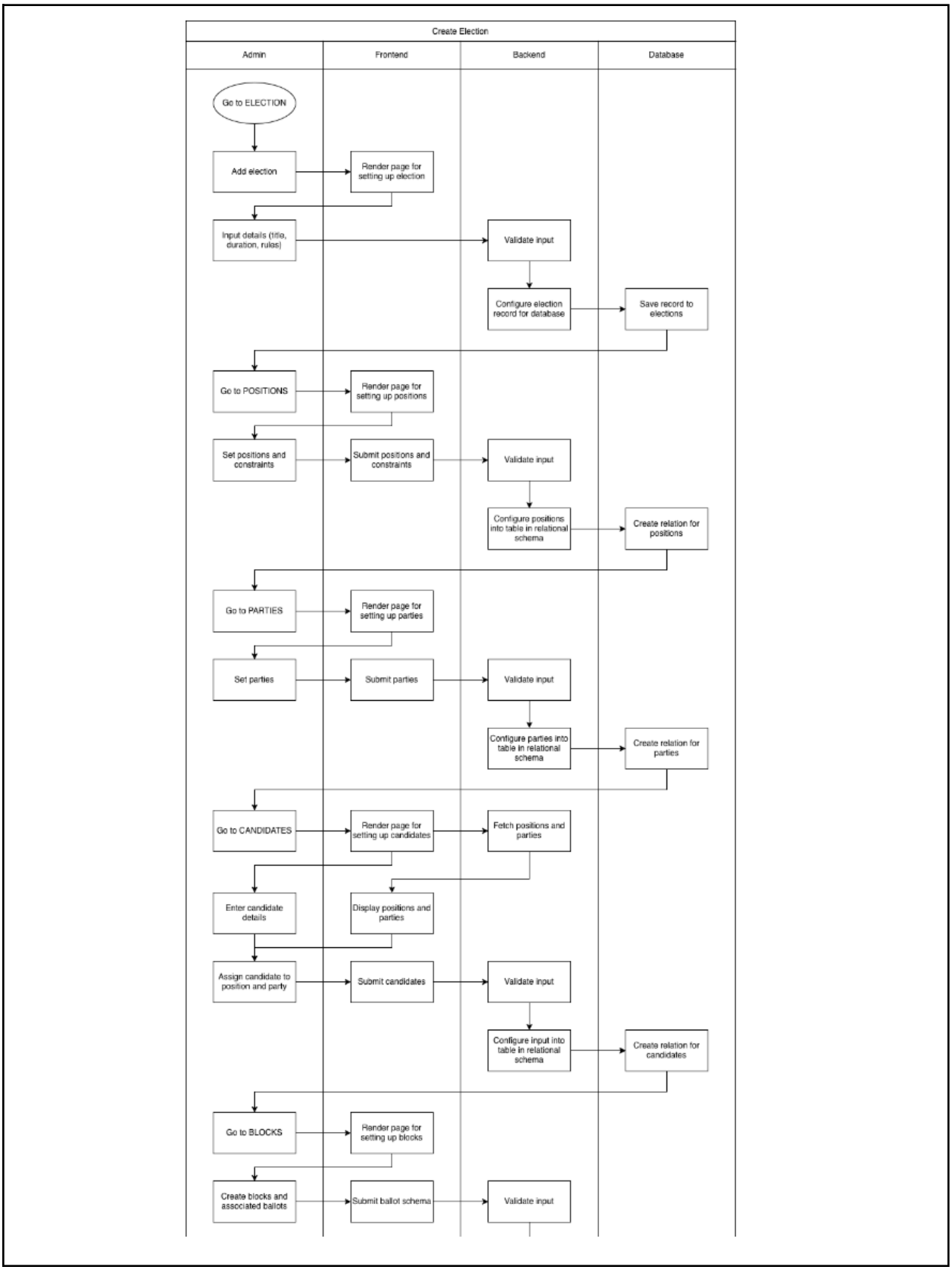
Create Election. An admin can create an election. Other than inputting the information and parameters for the election, other significant subroutines for the creation of elections are the creation of blocks which dictate the ballot of a certain subset of voters, the importing of the roster of registered voters, and the mapping of these voters to their respective blocks.

Start Election Proper. Once the election is scheduled to start, the admin must publish the election and set its status to “Active”. Afterwards, the system must disable admin permissions to edit any information or data related to the elections that might tamper with the results. This is a subroutine and the election can not proceed without this.

End Election Proper. Once the duration of the election has lapsed, an admin may officially end the election and set its status to “Inactive”. This will finalize all the results.

View Election Results and Statistics. Throughout the election and after it, an admin may view the running results and statistics.

System Sequence Diagrams



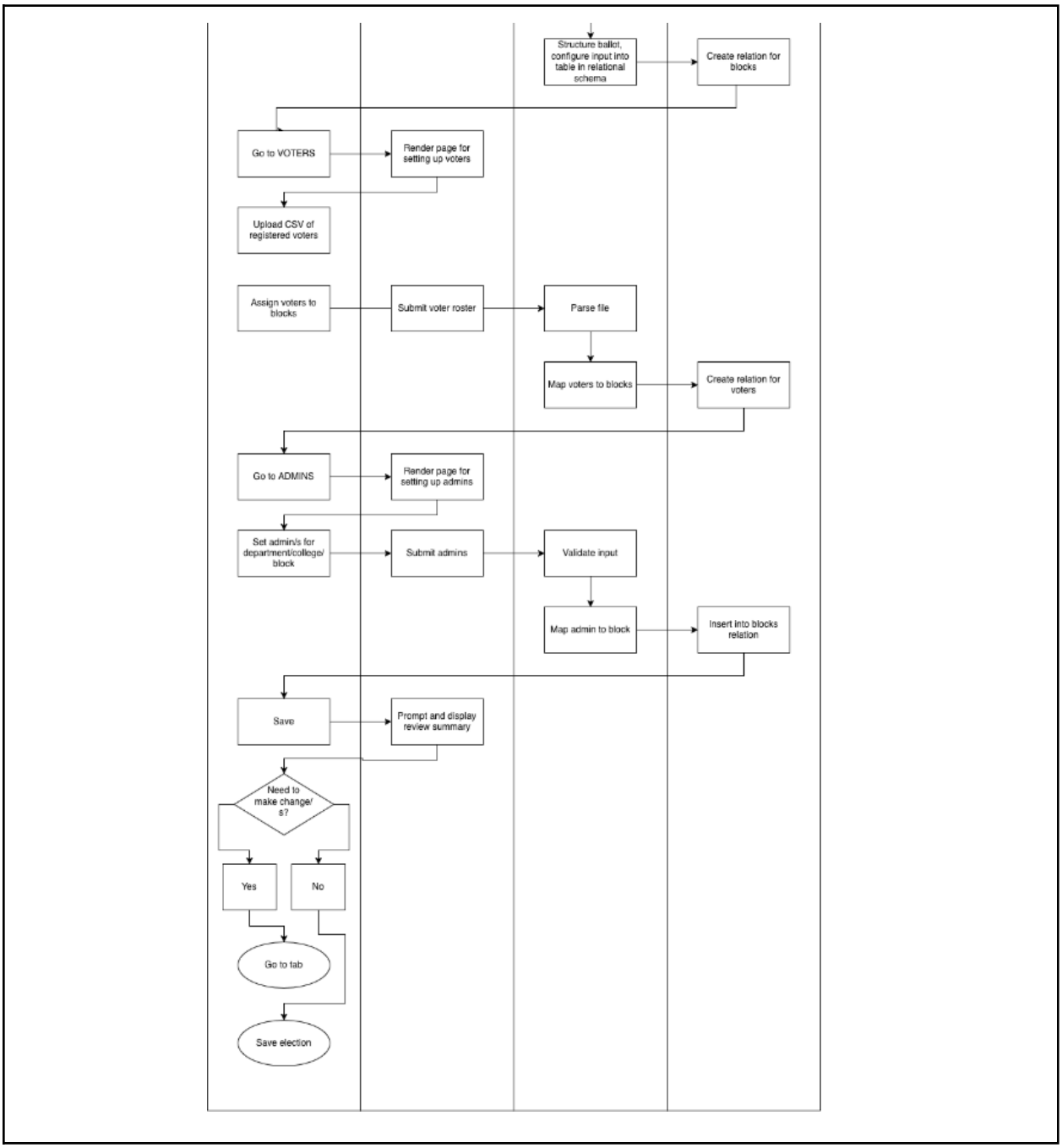


Figure X. Swimlane Diagram for Admin Use Case “Create Election”

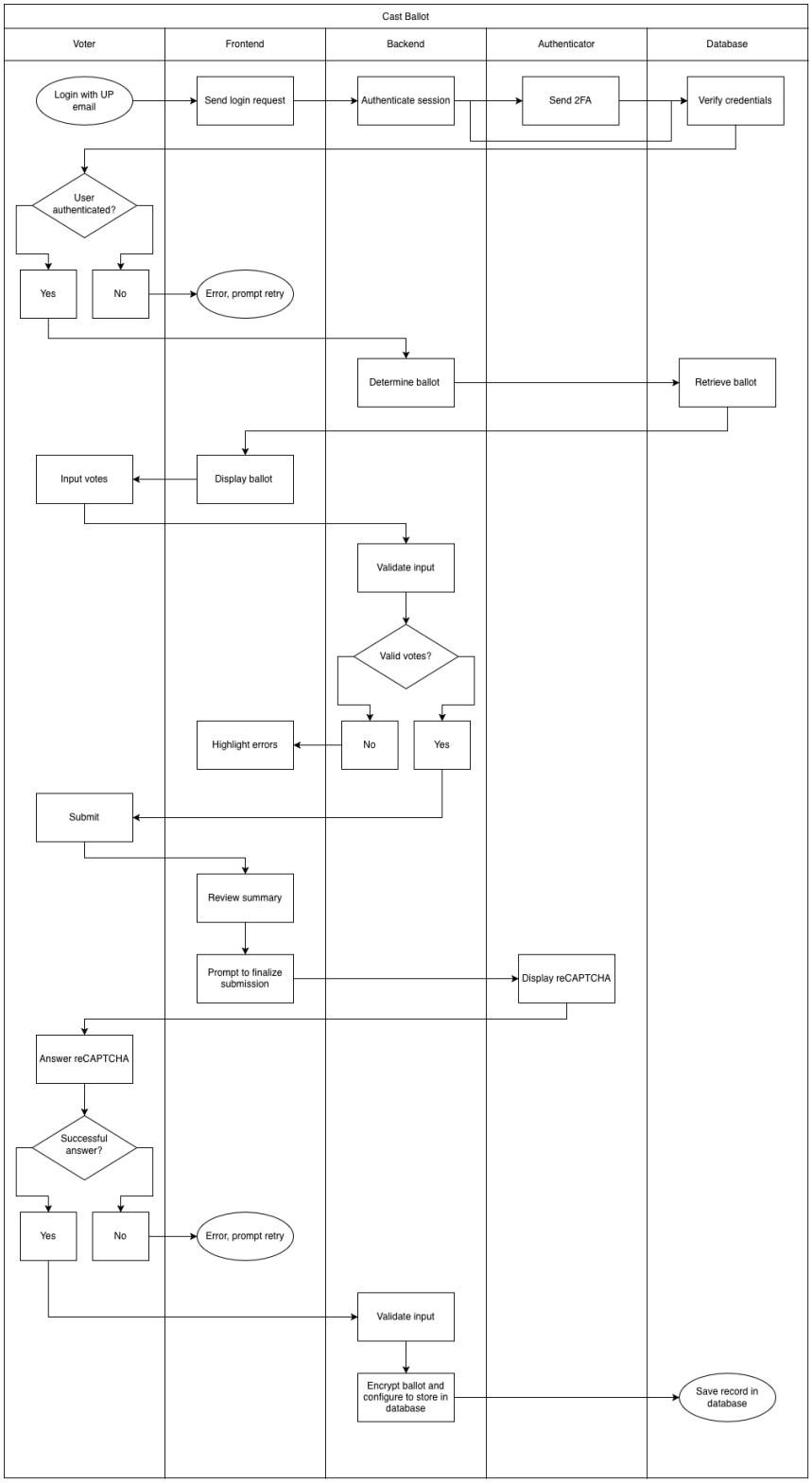


Figure X. Swimlane Diagram for Voter Use Case “Cast Ballot”

1.3 Configuration Model

This section specifies the allocation of software functionality and subsystems within the physical computing environment.

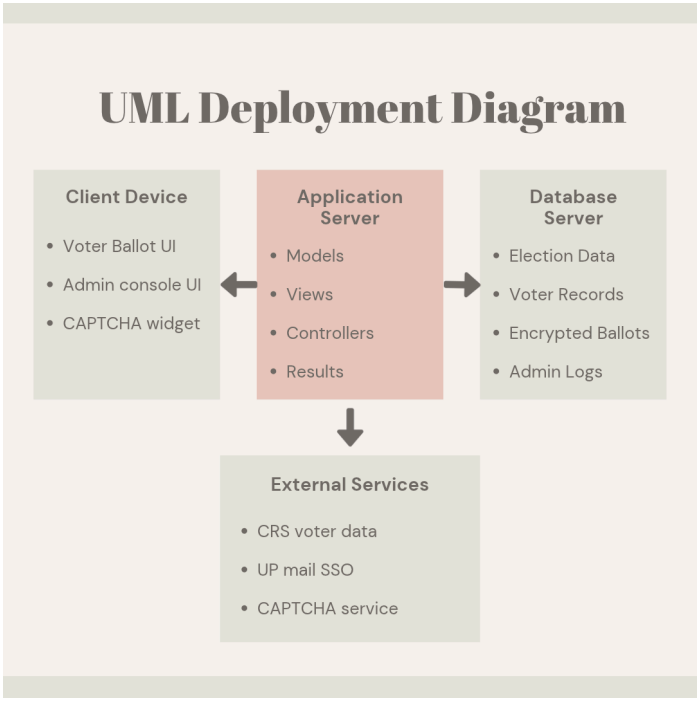


Figure 2. UML Deployment Diagram

The UML Deployment Diagram shown above has three main nodes: the client device, the application server, and the database server, with three external services needed below. The application server and database server can start on one VPS during development. They are separate nodes in the deployment model so the database can be isolated and scaled independently, and it should only accept connections from the application server.

2. Web App Architecture and Design

2.1 Architecture Design

Architecture Genre

Halalan fits the **Government** genre as defined by Grady Booch (cited by Pressman in Section 9.2), which covers systems that support the conduct and operations of a local, state, federal, global, or other political entity. As a voting platform, it is designed to support and conduct the

student elections of the University of the Philippines Diliman, which are part of the student government's electoral process.

Architecture Style:

Halalan uses a **layered** architecture style and applies the **Model-View-Controller (MVC)** pattern. The user interface layer contains the views that voters and administrators see. The application layer contains the controllers and models that implement election logic including block-based eligibility, vote and abstain handling, and results, together with authentication and validation services. The data layer holds the MySQL database.

Content Architecture

The **composite** content structure is the most appropriate for Halalan since voters and administrators follow different flows. The voter side uses a **linear structure**, moving from sign-in to choosing candidates, confirming selections and receiving a receipt. The administrator side uses a **hierarchical structure**, in which the content is split into main tabs and further options sit within each tab.

Quality Requirements Tree

Technical Attribute	Sub-attribute	Addressed by
Usability	Global site understandability	A consistent, uncluttered layout for administrators and a single guided path for voters.
	Online feedback and help features	Messages for failed sign-in and invalid input, and a confirmation step before a ballot is submitted.
	Interface and aesthetic features	A shared template for all pages, and a layout that works on most devices.
	Special features	A sample ballot per block and a printable ballot receipt

Functionality	Searching and retrieving capability	Results and statistics filtered by election and block.
	Navigation and browsing features	The sign-in page leads to the area for that user, and admin tabs reach each management section.
	Application domain-related features	Block-based ballot eligibility, abstention, nested elections and per-block results.
Reliability	Correct link processing	Role checks that send each user to their own area. (e.g voter can't open admin pages)
	Error recovery	Failed sign-in or an expired session returns the user to the Gate with a message.
	User input validation and recovery	The system checks every ballot before saving it, including the maximum votes per position, and rejects invalid ones with a message so the voter can fix them.
Efficiency	Response time performance	Pages are built on the server and sent ready to display, so voters get quick responses.
	Page generation speed	Totals such as the vote counts are calculated directly by the database in one step, instead of being added up one by one.
	Graphics generation speed	The pages use only small images, such as logos and candidate photos, so they load quickly.
Maintainability	Ease of correction	MVC separation. A fix only requires modifying an isolated part of the codebase
	Adaptability	Settings and private details such as passwords and keys

		are kept in one separate file, so they can be changed without touching the program itself. The sign-in method can also be switched between Google and username and password.
	Extensibility	Elections, positions, parties and blocks are stored as data, so adding a new one doesn't require changing the program. Changes to the code and database structure are documented properly.

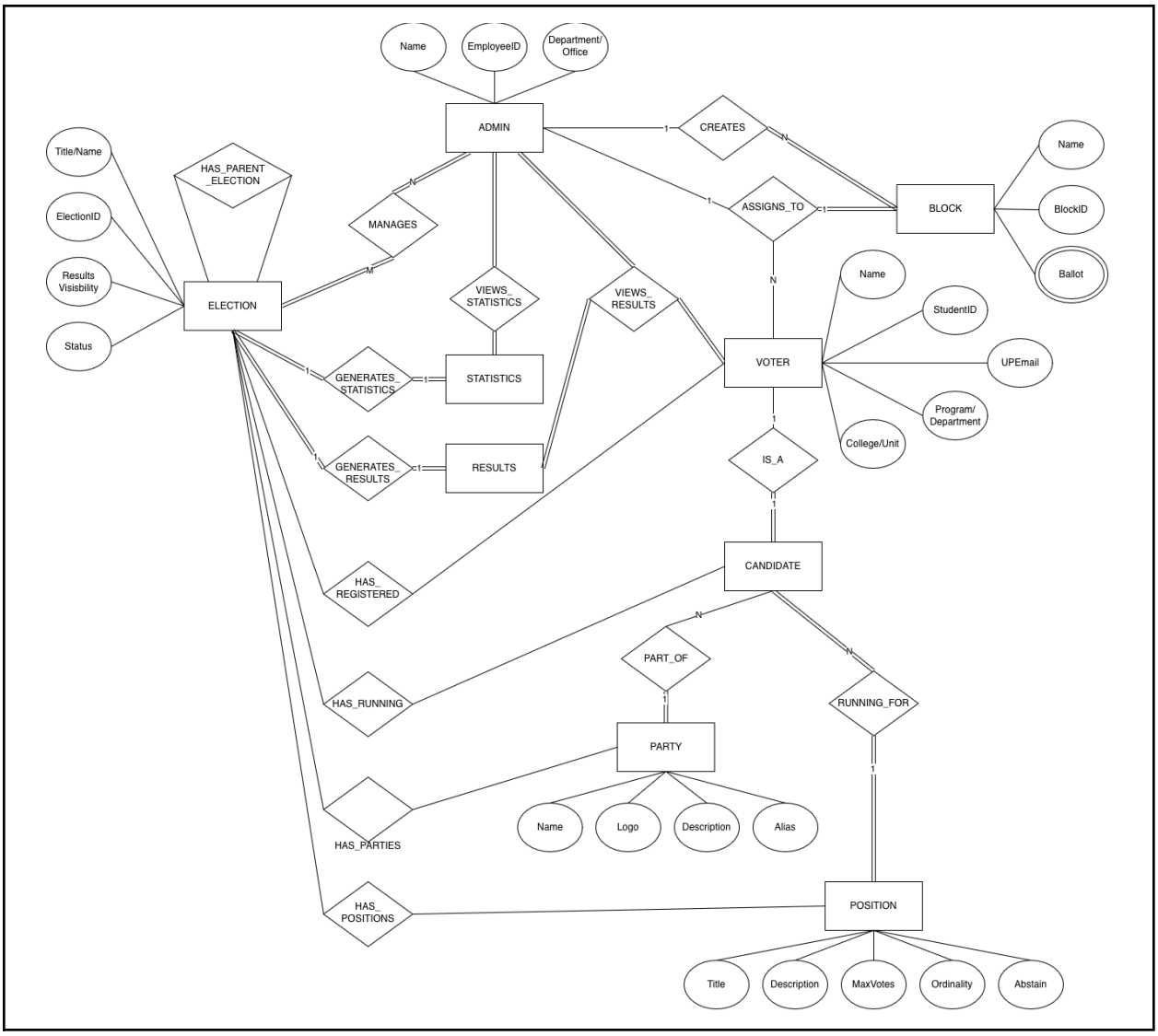
Architectural Decisions

Decision	Argument	Alternatives rejected
Rebuild on CodeIgniter(CI) 4	CI 4 is lighter than the alternatives.	Laravel
Layered architecture with MVC	The pattern is already familiar to the team.	Other patterns

2.2 Content (Data) Design

Class Diagrams

Entity-Relationship Diagrams



Data Flow Diagrams

2.3 Component-level design

The components are shown below:

Component	Processing	Interfaces	Dependency Type
Credentials Manager	Checks the person against the stored voter or admin	voter_login(), admin_login(), logout()	Flow

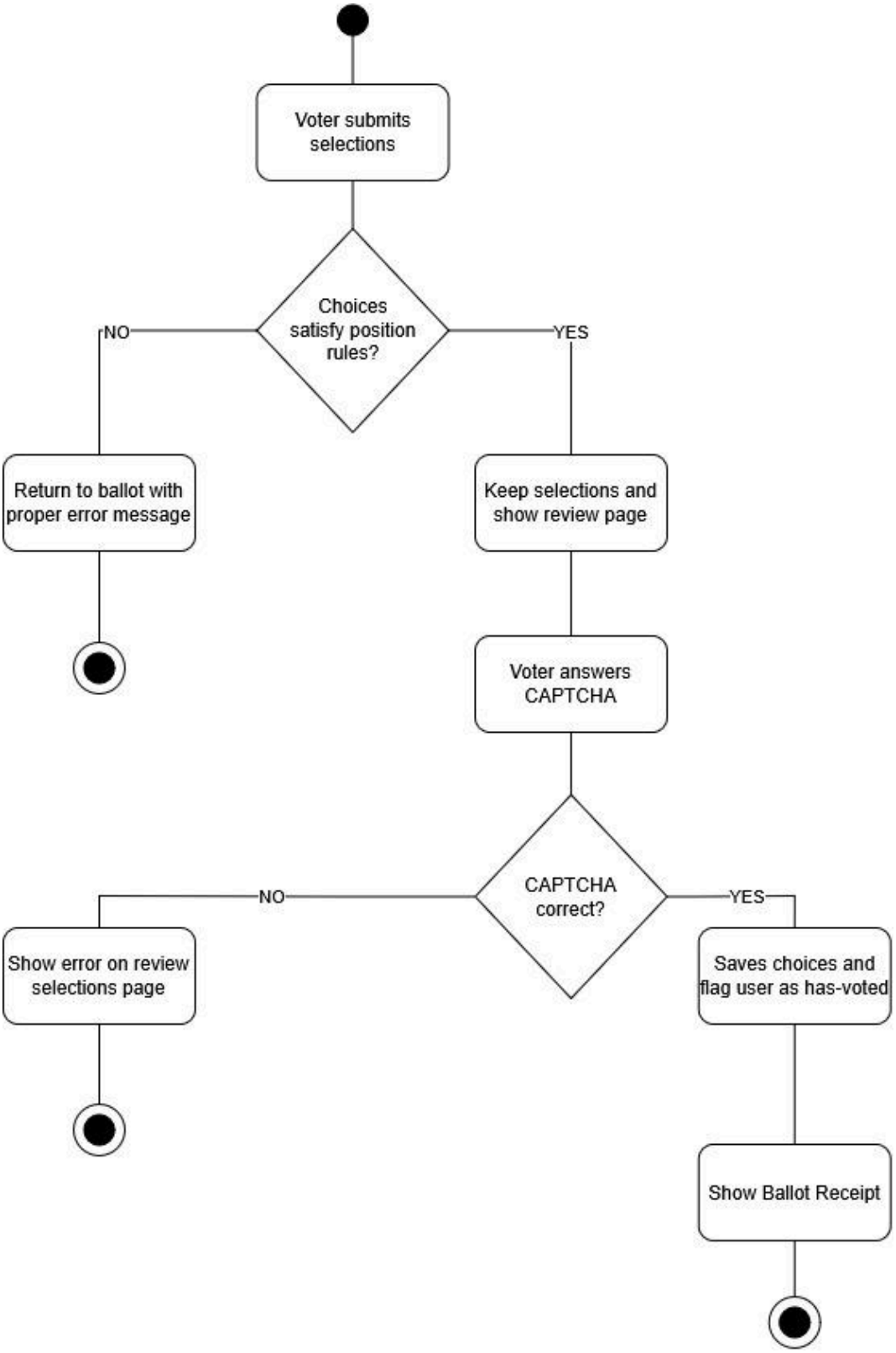
	credentials.		
Ballot Handler	Finds the voter's block, and loads the available choices that the voter is eligible for.	vote()	Sharing
Vote Recorder	Prompts the voter user for confirmation of their vote. On confirmation saves the votes, abstains and flags that the user has voted for no double counting	do_vote(), do_verify()	Flow, Constrained
Ballot Receipt Viewer	Collects the voter's own saved choices and displays it to the voter.	votes(election_id)	Flow
Election Manager	Creates, edits and deletes elections, positions, parties, candidates and blocks, and inserts the admin for that election. It rejects a change when an election is running.	insert() update() delete() insert_admin() (on Election object)	Sharing
Voter Users Manager	Adds, edits and deletes voters and lists them by block.	insert() update() delete() (on Voter object)	Sharing
Admin Users Manager	Lists the election administrators and generates a new password or key for each on request.	get_admins(), update(), update_key()	Sharing
Results Counter	Counts votes per candidate and abstains per position and also how many	count_votes() count_abstains() count_voters()	Sharing

	voters registered and voted.		
Activity Logger	Records user activity, and lists the entries for the Logs tab.	insert_log() get_logs()	Sharing

COTS

Component	Purpose
CodeIgniter 4	PHP Framework
MySQL	Selected DBMS
Google OAuth2 package	Verifying a voter's UP Mail through Google sign-in

Algorithms


Voter submits selections
Choices satisfy position rules?
NO
YES
Return to ballot with proper error message
Keep selections and show review page
Voter answers CAPTCHA
CAPTCHA correct?
NO
YES
Show error on review selections page
Saves choices and flag user as has-voted
Show Ballot Receipt

Data Structures

A nested key-value array holds the voter's selections (election, then position, then chosen candidates or abstain), a keyed array holds each position's maximum votes for checking the ballot, and simple lists hold the elections a voter has already voted in and the elections shown to them, with parents listed before their children. Elections form a tree, where each child election points to its parent. These were chosen for simplicity first, since the data is small (a campus-scale election), and then for fast lookup by id.

2.4 Interface elements / Usability design

This section describes the user interface of the Halalan revamp. The revamp keeps the flow of the current Halalan (Figures 1-5 in of the Project Plan) but with a cleaner, more modern look. Additional changes will also be made to account for what the requirements call for such as support for mobile devices, optimized admin interface, vote validation, CAPTCHA, and a more efficient results page. (* – means subject to change)

Interface Design - Menus and Links

a. **General Pages.**

Element/s	Layout	Target Page
Header	Displays the logo at the top left section and the tab bar.	–
Footer	Displays the text 'Powered by Halalan UP.'	–
Tab Bar	Horizontal tab bar heading the page that displays login/logged in as [name] and logout. Current tab open is marked with an underline.	Login and Logout Page
Banners	Rounded alert banners for a more modern look. Use of green banners for success messages and red banners for errors or notices.	–

b. **Voter Pages.**

Element/s	Layout	Target Page

Login Page	One central maroon button with the text ‘Sign in with your UP Mail.’ Other login methods for users outside of the university will appear as buttons below it.	UP Mail SSO
Ballot Page	One card per position, with each row signifying a candidate (name, party, photo*) and the last row being the abstain option if valid. Lastly, a submit button will be present at the bottom of the page.	Review and Confirm Page
Review and Confirm Page	A summary of all the chosen candidates per position, with Confirm and Back buttons at the bottom.	Receipt or back to Ballot Page

c. **Admin Pages.**

Element/s	Layout	Target Page
Modules Menu	Vertical menu listing HOME, ELECTIONS, POSITIONS, PARTIES, CANDIDATES, BLOCKS, VOTERS, ADMINS, LOGS, with the active module highlighted in maroon	Module List Pages
Home Menu	‘What do you want to do?’* card with a list of links containing the module list pages, password regeneration, and database reset	Module List Pages, Password Regeneration Page, Database Reset
Password Regeneration Page	A textbox for the username, a checkbox for ‘reset login time’, and a regenerate password button. After clicking the button, a feedback banner must appear to signify error or success.	
Database Reset Page	A button for database reset or delete with a confirmation pop-up. After clicking the button, a feedback banner must appear to signify error or success.	

Module List Pages	A table with the election information of the current module, with an add item option at the bottom.	Row Actions
Row actions	Pencil icon to edit, red cross to delete, 'stop' and 'hide' links next to the election status and results state. Order of rows can be modified using drag-and-drop.	–

d. Public Statistics Page

Element/s	Layout	Target Page
Statistics Page	Show the required statistics of the election in card and list form.	–

Aesthetic Design - Layout and graphics

Navigation Design

References

Pressman, R. S., & Maxim, B. R. (2010). *Software engineering: A practitioner's approach* (7th ed.). R.S. Pressman & Associates, Inc.